

해커톤 결과 보고서

다이어트 동아리 활동 기록 관리 웹앱 — 내 몸에 실제로 효과가 있었던 음식과 운동 찾기

팀번호 / 팀이름	1팀 / schoolLover	제출일	2026년 8월 5일
팀원 및 역할	이동제 — 프론트엔드 (index.html, record.html, style.css, js/ui/) · 박효민 — 데이터 계층 (js/store/, 설계서) · 봉준표 — 데이터 처리 (js/analytics/) / 공통: 디버깅 · 유지보수		
GitHub 저장소	https://github.com/bongjunpyo/hackathon1schoolLover		

1. 문제 정의와 기획

해결하려는 문제. 다이어트 동아리에서 활동 기록은 계속 쌓이는데 정작 중요한 질문에는 아무도 답하지 못한다 — “지난 한 달 동안 내가 살이 빠진 건, 뭘 해서 빠진 거지?” 기존 기록 앱은 “무엇을 했는가”까지만 남긴다. 러닝 20회, 헬스 15회, 총 1,200분. 숫자는 많은데 그중 어떤 게 실제로 체중을 움직였는지는 알 수 없다. 이 앱은 날짜별 체중 변화를 그날의 운동·음식에 귀속시켜 항목별 감량 효과 순위를 만든다. “닭가슴살을 먹은 날은 하루 평균 -0.115kg ”처럼 내 데이터로 검증된 문장을 만드는 것이 목표다.

주 사용자와 사용 상황. 다이어트 동아리 임원과 회원. 운동 직후 휴대폰이나 PC에서 활동명·날짜·장소·참여 인원과 함께 당일 체중·섭취 칼로리를 한 줄로 남기고, 한 달치가 쌓이면 통계 화면에서 추세를 본다.

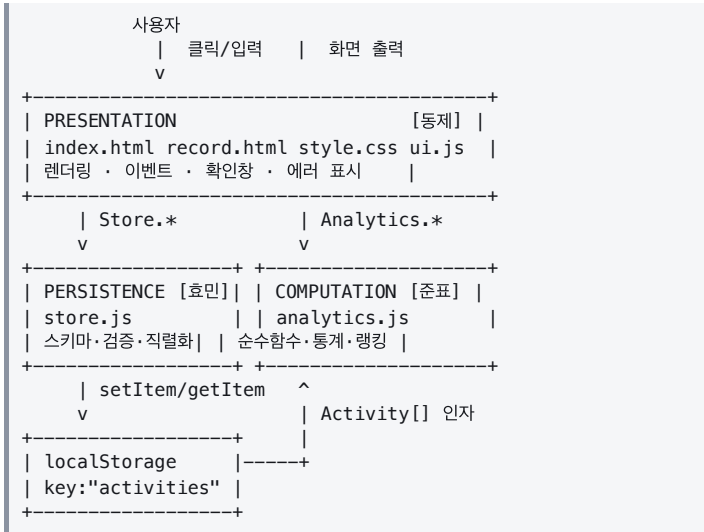
구현한 필수 기능 3가지. ① **활동 등록** — 활동명·날짜·장소·참여 인원·메모 입력 후 저장. ② **활동 목록 조회** — 최신순 정렬, 기록이 없으면 안내 문구 표시. ③ **활동 삭제** — 항목별 삭제, confirm() 확인 절차를 거친다. 검증 3중(활동명 빈값 거부 / 미래 날짜 거부 / 참여 인원 1 이상 정수)은 store.js 가 저장 직전에 최종 방어한다.

구현한 선택 기능과 선택한 이유

명세서 항목	구현	선택한 이유
월별 활동 횟수 통계 차트	Analytics.monthlyCount()	날짜가 이미 필수 필드라 추가 입력 없이 만들 수 있다
참여 인원 합계 및 평균	Analytics.summary()	참여 인원이 필수 필드다. 집계 한 번으로 끝난다
JSON 내보내기 / 가져오기	Store.exportJson() / importJson()	서버가 금지되어 기기 간 공유가 불가능한데, 이 기능이 그 한계를 우회하는 유일한 수단이다
반응형 레이아웃	style.css	운동 직후 휴대폰으로 기록하는 상황이 주 사용 시나리오다

명세서 목록 밖의 자체 확장으로 체중 추이, 칼로리 수치, 감량 효과 Top3와 회피 항목을 추가했다.

2. 설계



화면 구성

index.html 랜딩 · record.html 기록/통계 2화면. 기록 화면은 오늘의 체중 카드 + 활동 목록(최신순) + 등록 폼 + 통계 + 데이터 도구로 구성된다.

경계 규칙 — 동시 작업의 근거

계층	금지
ui.js	localStorage 직접 접근 (항상 Store 경유)
analytics.js	document · localStorage · Store 참조
store.js	alert · DOM 조작 (에러는 errors 객체로 반환)

COMPUTATION 계층이 아래를 향하지 않는다. analytics.js 는 저장소를 모르고 배열을 인자로 받기만 한다. 그래서 store.js 가 없던 시점에 배열을 손으로 만들어 통계 함수를 완성·검증했고, 세 명이 서로를 기다리지 않았다.

데이터 구조 — `localStorage["activities"]` 키 하나에 JSON 배열로 통째 저장

필드	타입	필수	제약	쓰이는 곳
명세서 고정 필드 — 삭제·개명 금지				
<code>id / createdAt</code>	string	자동	Store 발급, ISO 문자열	삭제 대상 지정, 최신순 정렬
<code>title</code>	string	O	공백 제거 후 1자 이상	목록 표시
<code>date</code>	string	O	YYYY-MM-DD, 미래 날짜 거부	월별 집계·체중 추이·랭킹
<code>place / memo</code>	string	—	제한 없음	목록 표시
<code>memberCount</code>	number	O	1 이상 정수	인원 합계·평균
확장 필드				
<code>category</code>	string	O	유산소·근력·스트레칭	종목별 집계, 칼로리 소모
<code>durationMin</code>	number	O	1 이상 정수	운동 시간 합계, 칼로리 소모
<code>weightKg / kcalIn</code>	number null	—	0 초과. 미입력은 <code>null</code>	체중 추이, 칼로리 수지, 랭킹
<code>exercise</code>	string null	—	러닝·헬스·수영·자전거·등산	운동 랭킹 집계 키
<code>foods</code>	string[]	—	12종에서 선택 (감량 7·증량 5)	음식 랭킹 집계 키

설계 결정 3가지

① 미입력은 `null` 로 통일. `undefined` 는 `JSON.stringify` 에서 키째 사라져 내보내기→가져오기 양복에 필드 유무가 달라진다. ② 빈 문자열은 `''` 이 아니라 미입력. `Number('')===0` 이라 그냥 넘기면 체중 0kg가 저장되고 평균이 무너진다. ③ `exercise·foods` 는 선택형 입력만. 랭킹의 집계 키라서 자유 텍스트면 "닭가슴살"과 "닭 가슴살"로 표본이 쪼개져 순위가 무너진다.

CRUD — 모든 변경은 Store 를 통과한다

```
Create  Store.add(input)  -> validate() 실패시 { ok:false, errors } 반환 (localStorage 변화 없음)
                                     -> buildActivity() id 발급·createdAt 기록·'' -> null 정규화 -> 전체 직렬화
Read    Store.getAll()   -> JSON.parse (깨진 값이면 []) -> createdAt 내림차순 -> Activity[]
                                     이 배열 하나가 목록·통계·랭킹의 단일 진실 공급원
Update  미구현 — 명세서 선택 기능이나 폼 재사용 리팩터 비용이 커서 잘라냄. 삭제 후 재등록
Delete  Store.remove(id) -> UI 확인창 -> 해당 id 제외 -> 길이 변화 확인 (없는 id면 false)
```

3. Claude Code 활용 과정

3-1. 작성한 CLAUDE.md

루트 1개(세 명 공통 계약) + 담당 폴더별 3개로 나눴다.

넣은 규칙	왜 필요하다고 판단했는지
인터페이스 계약 시그니처를 먼저 고정	상대 코드가 없어도 자기 몫을 끝낼 수 있어야 세 명이 동시에 작업한다
계층별 금지 사항 (경계 규칙 3줄)	파일 경계를 사람 경계와 일치시켰다. 5개 브랜치를 머지하는 동안 코드 충돌 0건
없는 값은 0으로 취급하지 않는다	<code>0/0 = NaN</code> 이고 <code>NaN</code> 이 화면에 찍히면 그대로 감점 스크린샷이 된다
서버·DB·빌드 도구 금지, <code>type="module"</code> 금지	명세 제약이자 <code>file://</code> 실행 전제. 못 박지 않으면 AI가 익숙한 방식(모듈·번들러)으로 돌아간다
커밋 본문에 검증 방법 한 줄	보고서를 쓸 때 기억에 의존하지 않아도 된다. 이 문서 대부분이 커밋 로그에서 나왔다

수정 이력. 14:05 역할 재정의(명세가 서버를 금지하므로 "백엔드"가 성립하지 않아 데이터 계층/데이터 처리로 분리) → 14:25 담당별 문서 분리 → 14:35 다이어트 중심으로 방향 전환(계약·스키마는 건드리지 않아 진행 중이던 작업이 멈추지 않았다) → 15:20 `exercise·foods` 2필드 추가(이슈 #2 결정) → 15:40 `weightTrend` 중복 처리 명시.

3-2. 구현 전 계획과 달라진 부분

- ML 기능의 정의가 바뀌었다. "날씨 기반 운동 추천"으로 잡았으나 실제로 원하는 것은 "내 몸에 효과가 있었던 음식·운동 Top3"였다 (3-4 참조)
- 날씨 기능을 계획에서 뺐다. 구현까지 마쳤으나 `file://` 에서 외부 `fetch` 가 막힐 수 있어 이슈 #2에서 B-2로 연기했다
- 활동 수정을 잘라냈다. 폼 재사용을 위해 등록 로직을 리팩터해야 해서 비용이 컸다
- 계획에 없던 이슈 트래커를 썼다. 계약 충돌이 구두로 정리되지 않아 GitHub 이슈에 선택지와 결정 시한을 적고 세 명이 코멘트로 결정했다

3-3. 핵심 프롬프트 3건

구분	무엇을 요청했나	어떤 결과를 받았나 / 만족했는지
① 확인	"운동동아리를 만들려고해. 헬스 운동을 기록하고 동아리원의 오늘 운동 부위 등을 볼 수 있는 알람 도 가계금. 앱이 나올까 웹이 나올까?"	앱/웹 답변보다 먼저 두 기능이 명세 제약에 걸린다 는 지적. (1) 운동 부위 공유는 localStorage가 브라우저 하나에 갇혀 있어 서버가 필요한데 명세가 금지 (2) 브라우저 알림은 <code>file://</code> 에서 권한 요청 자체가 차단. 만족 . 그대로 만들었다면 시연 중에 동작하지 않는 기능 두 개를 꺼내고 갔을 것이다
② 준표	"ml작업할 건데 (...) 한 달동안 어떤 운동을 하였고, 어떤 식단을 하였으며, 얼마나 몸의 변화가 일어났는지 분석하고 (...) 다이어트 동아리야. 웹으로 구현할거야."	계약 총돌 3건을 먼저 지적(백엔드 역할 부재 / 데이터 0건이라 샘플 생성 필요 / 스키마는 확장만 가능). ML 방식으로 TensorFlow.js 대신 순수 JS 최소자승 회귀 + 피어슨 상관을 권고 — 외부 의존이 0이라 데모 중 터지지 않는다. 만족 . <code>analytics.js</code> 가 <code>Store</code> 를 모르게 설계한 덕분에 <code>store.js</code> 가 없던 시점에 통계 함수를 전부 완성·검증했다
③ 확인	"이번 주 다이어트 계획 을 입력한다"를 컨셉으로 잡고 진행 요청	명세서 검증 요건에 " 날짜는 미래 날짜 입력 불가 "가 있어 계획(=미래 날짜)과 정면 충돌한다는 지적. 만족 . 컨셉을 "계획 입력"에서 " 기록 후 분석 "으로 돌렸고, 지금의 정체성이 여기서 나왔다

3-4. 프롬프트 개선 사례

처음: 날씨에 따라 그림 사람들이 어떤 스포츠를 선호하는지도 찾아서 자세히 어떤 운동을 어떤 시간대에 하면 좋은지도 같이 분석해서 덧붙이면 좋을 것 같아.

기대와 달랐던 지점. 요청대로 **날씨를 중심에 둔 기능**이 나왔다. 예보로 시간대를 점수화해 최적 2시간 구간을 고르는 엔진이었고 동작은 했지만 방향이 어긋났다. 주인공이 날씨가 되어 있었고, 운동 분류가 카테고리 3종 단위라 "러닝이 효과적"이 아니라 "유산소가 효과적"까지밖에 못 나왔다. **식단 분석은 아예 없었고**(칼로리 총량만 있고 음식 종류가 없었다) 추천도 오늘 하루치뿐이었다.

고친 뒤: 지금 날씨로 예측 ml이 주요 기능이 아니야. 내 몸이 어떤 음식과 어떤 운동에서 가장 체중 감량이 많이 되었고, 그 중에서도 top3를 매겨 다음 날짜들에 맞춰서 날씨 기준으로 잡고 예측을 해주는 거야.

무엇이 본체이고 무엇이 보조인지 명시했다. 처음 요청은 "~도 덧붙이면"이라는 추가 요청 형태여서 중심이 빠져 있었다.

	처음	고친 뒤
본체	날씨 기반 시간대 추천	개인 효과 랭킹 (날씨는 배치 필터로 강등)
분석 단위	카테고리 3종	개별 운동 5종 + 개별 음식 12종
식단	없음	foods 필드 추가, 음식별 감량 기여도 산출
계산 모델	주간 상관분석	귀속(attribution): $\Delta w(d) = (\text{다음 기록일 체중} - \text{당일 체중}) / \text{경과일수}$

검증용 데이터에 러닝 -0.15kg/일, 헬스 -0.08, 요가 +0.05를 심어두고 돌렸더니 **심어둔 값을 소수점 셋째 자리까지 정확히 복원**했다. 두 운동일에 걸친 닭가슴살이 두 값의 평균(-0.115)으로 나온 것까지 맞았다. 얻은 교훈은 "**무엇을 추가할지**"보다 "**무엇이 중심인지**"를 먼저 **말해야 한다**는 것이다. 추가 요청 형태로 말하면 시는 직전 맥락을 중심으로 삼는다.

3-5. 오류 해결 사례

발생한 오류. 16:26, 마감 14분 전. 세 사람의 브랜치를 합치고 화면을 처음 열었더니 **필수 3기능이 하나도 렌더되지 않았다**. 목록에는 "저장 모듈(store.js)을 기다리는 중입니다", 폼은 비어 있고 통계는 `hidden`. 그런데 콘솔 에러는 `favicon.ico 404` 하나뿐이었다 — **에러 없이 조용히 실패하는 상태**였다.

원인 파악 과정. 로드 순서와 `type="module"` 미사용을 먼저 확인했다(정상). 다음으로 콘솔에서 모듈 정의를 확인했다.

```
typeof Store -> "object"      정의되어 있다      window.Store      -> undefined      <-- 여기
typeof Analytics -> "object"   정의되어 있다      window.Analytics   -> undefined

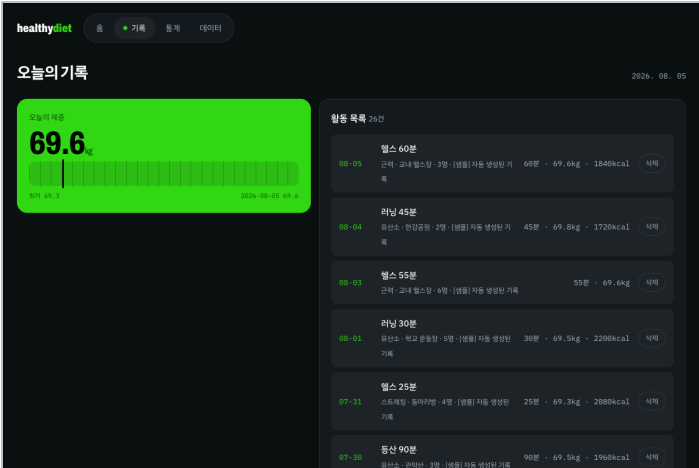
// ui.js 가 모듈 존재를 확인하는 방식
function ready(name) { return typeof window[name] !== 'undefined' && window[name]; }
```

`store.js`와 `analytics.js`는 둘 다 최상위 `const`로 선언한다. 최상위 `const·let`은 전역 렉시컬 스코프에 들어갈 뿐 **window의 프로퍼티가 되지 않는다**. `var`나 함수 선언이었다면 붙었을 것이다. 그래서 `ready()`가 항상 `false`를 반환하고 `render()`가 첫 줄에서 빠져나갔다.

해결. 각 모듈 끝에 `window.Store = Store; / window.Analytics = Analytics;` 한 줄씩 추가했다. CLAUDE.md가 "전역 객체로 노출한다"고 한 계약을 실제로 만족시키는 방향이라 `ui.js`를 고치는 쪽보다 변경이 작았다.

배운 점. ① "**에러가 없다**"가 "**정상이다**"는 아니다 — 존재 확인이 조용히 실패하면 아무 흔적도 남기지 않는다. ② **세 파일 모두 자기 기준으로 계약을 지켰다** — 전역이라는 단어의 해석이 달랐을 뿐이다. 계약은 문장이 아니라 실행되는 코드로 확인해야 한다. ③ **통합 확인을 마지막에 몰면 남는 시간이 없다** — 모듈 단위 테스트는 셋 다 통과하고 있었고, 세 파일이 한 페이지에서 만나야만 드러나는 종류였다. 같은 성격의 사례가 하나 더 있었다. 통합 테스트에서 **감량 랭킹이 정확히 반대로** 나왔는데(감량 1위 피자), `analytics.js`는 "기록 이후의 체중 변화를 귀속"시키는데 `seedSampleData()`는 오늘의 선택을 **오늘 체중에 이미 반영**해 저장했기 때문이었다. 증상이 "틀린 값"이 아니라 "정확히 반대"였던 것이 방향 문제라는 단서였다.

4. 실행 결과



화면 1 — 기록 화면. 오늘의 체중 카드와 추이 스파크라인, 활동 목록 26건(최신순), 각 항목에 날짜·활동명·종목·장소·인원·운동시간·체중·칼로리와 삭제 버튼. 기록이 0건이면 "아직 기록이 없습니다. 오늘 체중부터 남겨보세요." 안내가 대신 표시된다.



화면 2 — 통계. 활동 수 26 · 참여 인원 합계 103 · 평균 4.0 · 운동 시간 1290분 · 평균 50분, 월별 활동 횟수, 종목별, **감량 효과 Top3(운동·음식)**와 **피해야 할 항목**. 증감은 색이 아니라 방향 삼각형 (▼ 감량 / ▲ 증량)으로 표시하고 표본 수를 함께 적는다.

테스트한 항목과 결과 — 병합된 main 을 브라우저에서 직접 열어 확인

구분	항목	결과
필수 기능	등록 / 목록 / 삭제	저장 26→27건, 최신순 렌더 정상, 빈 목록 안내 정상, 확인창 후 삭제 27→26건
입력값 검증	활동명 빈값	거부
	미래 날짜	거부 — <code>input max</code> 로 1차, <code>store.js</code> 로 2차 방어. 오늘 날짜는 정상 저장
	참여 인원 0-~1·2·5·""·"abc" / 빈 문자열 체중	전부 거부 / 0 이 아니라 <code>null</code> 로 저장
선택 기능	인원 합계·평균 / 월별 활동	103 / 4.0, 2026-07 22회 · 2026-08 4회
	JSON 왕복 / 깨진 입력	26→26건, 필드 집합 동일, <code>kcalIn:null</code> 6건 보존 / <code>throw</code> 없이 <code>{ok:false}</code> , 실패는 <code>skipped</code> 집계
분석 엔진	감량 효과 Top3 / 회피 항목	운동 자전거·등산·수영, 음식 계란·현미밥·닭가슴살 / 치킨·탄산음료·피자
화면 오류	<code>NaN</code> · <code>undefined</code> 노출 / 콘솔	없음 / <code>favicon.ico</code> 404 하나뿐

5. 한계와 개선 방향

시간 내 구현하지 못한 것. ① 날씨 기반 7일 계획(`recommend.js`)은 구현·검증했으나 이슈 #2에서 B-2로 연기해 BJP 브랜치에 남았다. ② 활동 수정과 ③ 키워드 검색은 잘라냈다.

마감 후 보완 — 제출 시점에는 감량 효과 Top3가 화면에 표시되지 않았다. 엔진은 완성·검증됐지만 `ui.js` 가 `Analytics.topFoods()` 를 호출하지 않아 차별화 기능이 사용자에게 닿지 않는 상태였다. 총평 이후 렌더 블록을 추가해 현재는 통계 화면에서 확인할 수 있다 (PR #11).

현재 코드의 불안한 부분. **감량 효과는 인과가 아니라 상관이다** — 같은 날 여러 항목을 하면 효과가 나뉘어 귀속되고 표본이 적으면 우연이 순위에 섞인다(그래서 표본 수를 함께 표시하고 3건 미만은 제외했다). **칼로리 소모는 분 × 카테고리 계수로만 계산한 거친 추정치**라 절대값을 신뢰할 수 없다. 전역 객체 3개에 의존해 이름이 겹치면 조용히 덮이고, 실제로 이번이 그 사고가 났다. 브라우저 하나에만 저장되어 기기 간 공유가 안 되며, **자동화 테스트가 없어 회귀를 막을 장치가 없다**.

시간이 더 있었다면. ① **통합 확인을 15:00에 한 번 강제한다** — 이번 두 건의 오류(랭킹 반전, `window` 노출 누락)는 모달이 만나야만 보이는 종류였고 둘 다 마감 직전에 발견됐다. ② `recommend.js` 를 합류시킨다. ③ 서버를 붙여 동아리원 간 공유를 만든다.

6. 역할 분담과 소감

팀원	담당	주요 산출물
이동제	프론트엔드	<code>index.html</code> , <code>record.html</code> , <code>style.css</code> , <code>ui.js</code> , <code>landing.js</code> . 루트 및 담당별 <code>CLAUDE.md</code> 초안. 외부 CDN 실패 시 폴백 처리
박효민	데이터 계층	<code>store.js</code> (계약 7종·검증·샘플 생성·JSON 입출력), <code>설계서.md</code> , 열린 결정 5건 확정, 이슈 #2 최종 결정
봉준표	데이터 처리	<code>analytics.js</code> (계약 5종 + 랭킹 6종), <code>recommend.js</code> , <code>README.md</code> , 통합 검증 및 마감 직전 블로커 진단

이동제 — 규칙 파일을 먼저 쓰는 것이 생각보다 큰 차이를 만들었다. `CLAUDE.md` 에 " `type="module"` 금지, "서버 도입 금지"를 못 막아두지 않았다면 AI는 익숙한 방식으로 돌아갔을 것이고, 그때는 `file://` 에서 백지 화면을 보고 나서야 알았을 것이다.

박효민 — AI가 가장 크게 기여한 지점은 코드를 빨리 짜준 것이 아니라 **요청이 명세와 충돌한다고 먼저 말해준 것이었다**. "운동 부위 공유", "알람", "계획 입력" 세 가지 모두 그 대로 만들었다면 시연 중에 동작하지 않는 기능을 꺼안고 갔을 것이다. 반대로 아쉬운 것은 모달이 각자 맞아도 **모델이 어긋나면 통합에서 터진다**는 점이다. 랭킹이 정반대로 나온 문제는 두 파일이 만나야만 보였다.

봉준표 — 프롬프트에서 **무엇이 본체인지 말하는 것이 결정적**이었다. "날씨에 따라 ~도 덧붙이면"이라고 요청했을 때는 날씨가 주인공인 기능이 나왔고, "날씨 예측이 주요 기능이 아니야"라고 고치자 완전히 다른 설계가 나왔다. AI는 직전 맥락을 중심으로 삼기 때문에 추가 요청 형태로 말하면 우선순위가 뒤바뀐다. 그리고 **"에러가 없다"가 "정상이다"는 아니라는 것**을 마감 14분 전에 배웠다.